Predicting the Remaining Useful Life of Turbofan Engines

Phase 1: Sensor Feature Engineering with Classical Machine Learning
NASA C-MAPSS FD001 dataset
Team: Keshav, Rohit Kumar, Mayank Yadav, Arun, Sarvesh Kumar

1. Introduction

Aircraft engines wear out gradually. If maintenance is done too late the engine can fail in service, and if it is done too early useful life is thrown away. **Predictive maintenance** tries to schedule work at the right moment by estimating the **Remaining Useful Life (RUL)**: the number of operating cycles (flights) an engine can still complete before it reaches failure. In this project we treat RUL estimation as a **regression task**: from an engine's sensor history up to today, the model outputs a single number of cycles. As an example, an engine that fails at cycle 240 and is now at cycle 180 has an RUL of 60.

Phase 1 deliberately uses only classical machine learning. The guiding research question is: *can carefully designed degradation features allow traditional regression models to estimate engine RUL accurately?*

2. Aims of the Project

- Inspect the FD001 data and remove sensors that carry no information.
- Construct a reliable RUL label for every training row.
- Turn noisy raw readings into degradation features without using any future information.
- Tune and compare a Baseline, Linear Regression, a Decision Tree and a Random Forest on engines the models have never seen.
- Study where the models make their biggest mistakes, and score the chosen model once on NASA's official test engines.

3. Data and Software

We use subset **FD001** of NASA's C-MAPSS simulation (one flight condition, one fault mode: high-pressure compressor wear). Each row is a snapshot of one engine at one cycle and has 26 columns: engine number, cycle, three operating settings and 21 sensors.

File	Content	Role in this project
train_FD001.txt	20,631 rows, 100 engines run until failure	Model training and validation
test_FD001.txt	13,096 rows, 100 engines stopped before failure	Final, one-time evaluation
RUL_FD001.txt	100 numbers, one per test engine	True remaining life at each test engine's last cycle

Software: Python, pandas, NumPy, scikit-learn, Matplotlib/Seaborn, Jupyter Notebook and Streamlit (web app). All random processes use a fixed seed (42), so every number in this report can be reproduced.

4. Method

4.1 Data checks and sensor screening. The files contain no missing values and no duplicate rows. Engine lifetimes range from 128 to 362 cycles (mean 206). Counting distinct values per sensor showed that s1, s5, s10, s16, s18 and s19 are completely flat and s6 takes only two values, so these seven sensors were dropped. Operating setting 3 is fixed at 100 and the other two settings barely move, which confirms a single operating condition; therefore no condition-based normalisation was required and the settings were not used. Fourteen sensors remain.

4.2 Target construction. For every training engine, RUL = (final cycle of that engine) minus (current cycle), so it reaches zero at failure. Healthy engines show almost no sensor change, which makes it impossible to tell 300 remaining cycles from 150. We therefore cap the label at 125 cycles, a common choice for this dataset, and use the same capped definition for every model and for the test set.

4.3 Engine-wise validation split. Consecutive rows from one engine are nearly identical, so a random row split would leak information. Instead, whole engines were assigned: 80 engines for training and 20 for validation, with a check that no engine appears in both groups.

4.4 Degradation features. Every feature is computed inside a single engine using only the current and earlier cycles, over a 10-cycle window (shorter at the very start of an engine's history). For each of the 14 sensors:

Feature	How it is calculated	What it captures
Rolling mean	Average of the last 10 readings	Smoothed sensor level (noise removed)
Rolling std	Standard deviation of the last 10 readings	How unstable the sensor has become
Rolling slope	Slope of a least-squares line through the last 10 readings	Direction and speed of the trend
Drift from healthy state	Rolling mean minus the engine's own average over its first 10 cycles	How far this particular engine has moved from its normal level

Together with the cycle number and the 14 raw readings this gives **71 input features**. A leakage test rebuilds the features of an engine from a shortened history and confirms the values are identical, proving that no future readings are used.

4.5 Models and tuning

- **Baseline:** always predicts the mean training RUL; any useful model must clearly beat it.
- **Linear Regression:** ordinary least squares on standardised features (scaling inside a pipeline); prediction is a weighted sum of the features.
- **Decision Tree:** a CART regression tree that splits on the feature and threshold reducing the squared error most; growth is limited by pre-pruning (*max_depth*, *min_samples_leaf*) and cost-complexity post-pruning (*ccp_alpha*). Chosen: depth 12, at least 50 rows per leaf.
- **Random Forest:** averages 150 decision trees, each grown on a bootstrap sample of the rows (bagging). Chosen: depth 20, at least 5 rows per leaf.

Settings for each model were chosen by grid search with 3-fold **GroupKFold** over the 80 training engines, so one engine is never in two folds. The 20 validation engines were only used to compare the tuned models, and the model with the lowest validation RMSE was selected before the test set was opened. Performance is reported with RMSE (penalises large misses), MAE (average miss in cycles) and R^2 (share of variation explained).

5. Results

5.1 Validation comparison (all cycles of the 20 unseen validation engines):

Model	RMSE (cycles)	MAE (cycles)	R^2
Baseline (mean)	41.72	36.93	0.00
Linear Regression	15.29	11.94	0.87
Decision Tree	14.13	9.18	0.89
Random Forest	10.34	6.77	0.94

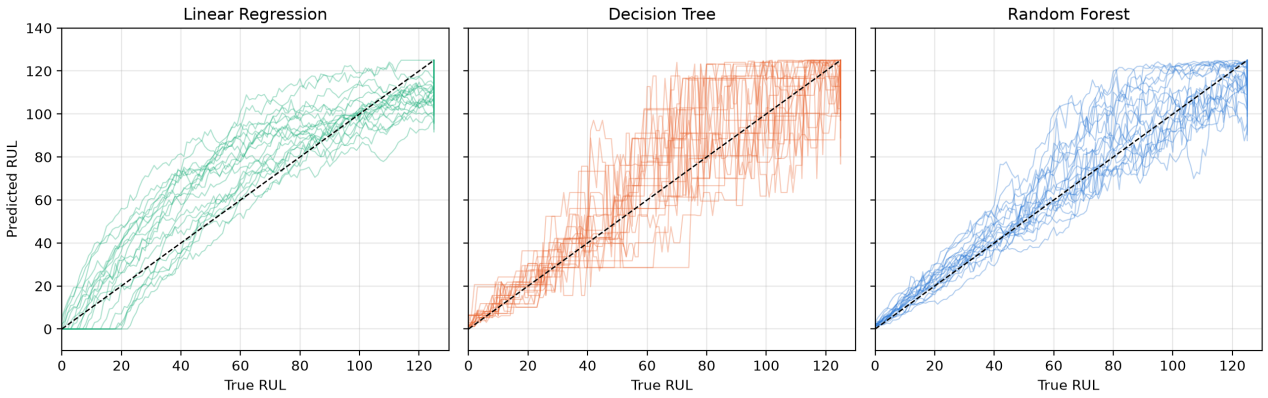


Figure 1. Predicted vs. true RUL on the validation engines, one line per engine (dashed line = perfect prediction).

5.2 Official NASA test set. Random Forest, selected on validation, was retrained on all 100 training engines. Features were built from each test engine's own history and one prediction was made at its last recorded cycle.

Model	Test RMSE	Test MAE	Test R^2
Baseline (mean)	41.94	34.83	-0.10
Linear Regression	15.85	12.84	0.84
Decision Tree	14.96	10.80	0.86
Random Forest (selected)	11.44	8.27	0.92

On average the Random Forest forecast is about **eight cycles** away from the true remaining life; 68 of the 100 test engines are predicted within 10 cycles and 89 within 20. The test error is close to the validation error for every model, which indicates that the engine-wise validation gave a trustworthy estimate.

5.3 Effect of feature engineering. Each tuned model was retrained on raw readings only (cycle + 14 sensors):

Model	RMSE, raw sensors only	RMSE, engineered features	Reduction
Linear Regression	17.37	15.29	12 %
Decision Tree	17.73	14.13	20 %
Random Forest	15.98	10.34	35 %

The feature-importance analysis shows that the **drift-from-healthy-state** features carry most of the predictive power: about 84 % of the Random Forest importance, followed by the rolling means (about 13 %). The single most important feature is **s11_dev**, the drift of sensor s11 (HPC static pressure), with about 74 %; next come s17 (bleed enthalpy), s4 (LPT outlet temperature), the cycle number, s15 (bypass ratio) and s9 (core speed). Rolling std and slope features add little once the drift is known.

5.4 Error by life stage (validation; early = more than 100 cycles left, late = 100 or fewer):

Model	Early RMSE	Late RMSE	Early MAE	Late MAE
Linear Regression	12.23	17.86	9.67	14.24
Decision Tree	10.01	17.33	5.65	12.75
Random Forest	8.01	12.26	4.97	8.59

Errors are larger late in life, when degradation speeds up and engines diverge from each other; for all three models the error peaks when 50 to 100 cycles remain. The low early-life error is partly a result of the 125-cycle cap: a healthy-looking engine is simply predicted near 125. The real difficulty is the period when

wear is just beginning to appear in the sensors.

6. Discussion and Limitations

- Random Forest gave the best scores on every metric, about 3.5 cycles of test RMSE better than the single Decision Tree; averaging many bootstrapped trees lowers the variance of one tree.
- The Decision Tree overfits easily: training error keeps falling as the tree grows deeper while validation error stops improving after a depth of about 7 to 10, so pruning was needed.
- Linear Regression is the easiest model to explain but cannot follow the curved, accelerating degradation pattern (flat, then a fast drop).
- Rolling windows react with a short delay to sudden changes, and test engines with short histories give less reliable trend estimates.
- Only FD001 (one operating condition, simulated data) was studied, with a single train/validation split.
- The models give point estimates without uncertainty ranges, which real maintenance planning would need.

7. Conclusion and Future Work

A clear, leakage-free pipeline of data checks, a capped RUL target, engine-wise validation and simple degradation features allowed classical models to estimate turbofan RUL with an **RMSE of 11.44 cycles** (MAE 8.27, R^2 0.92) on the official test engines, less than a third of the baseline error (41.94). Engineered features, especially each sensor's drift from its engine's healthy level, cut the Random Forest error by 35 % compared with raw readings, so the answer to our research question is **yes**. A Streamlit web app loads the final Random Forest and returns a predicted RUL and health status from entered sensor values or from any of the 100 NASA test engines.

Next steps: extend the work to the multi-condition subsets FD002-FD004, use repeated cross-validation for confidence intervals, add uncertainty estimates, and in Phase 2 compare these results with sequence-based deep learning models (e.g. LSTM) under the same split and target definition.

8. Team Members

Student Name	Enrollment Number	Email
Keshav	2401010221	keshav.2024@nst.rishihood.edu.in
Rohit Kumar	2401010392	rohit.kumar2024@nst.rishihood.edu.in
Mayank Yadav	2401010271	mayank.yadav2024@nst.rishihood.edu.in
Arun	2401010098	arun.2024@nst.rishihood.edu.in
Sarvesh Kumar	2401010422	sarvesh.kumar2024@nst.rishihood.edu.in

9. References

- Saxena, A., Goebel, K., Simon, D. and Eklund, N. (2008). Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation. International Conference on Prognostics and Health Management (PHM08).
- NASA Prognostics Center of Excellence. C-MAPSS Turbofan Engine Degradation Simulation Data Set.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- Breiman, L., Friedman, J., Olshen, R. and Stone, C. (1984). Classification and Regression Trees. Wadsworth.
- Breiman, L. (2001). Random Forests. Machine Learning, 45, 5-32.